Exp No: 6	**Bidirectional Recurrent Neural Network (RNN) for Time Series Prediction and Comparison with a Feed-Forward Network**

AIM:

To implement a bidirectional RNN to predict sequences in time series data, train it, and compare its performance with a traditional feed-forward neural network on a sequence-based task.

ALGORITHM / PROCEDURE:

1. Load the Airline Passengers series (144 monthly totals, 1949-1960) and scale it to 0-1 using the minimum and maximum of the first 120 months only.
2. Create samples with a sliding window: the input is 12 consecutive months and the target is the next month. The first 120 months give 108 training windows and the last 24 months give 24 test windows.
3. **Bidirectional RNN:** a forward tanh RNN reads the window from oldest to newest and a backward tanh RNN reads it from newest to oldest (16 units each). Their final states are concatenated and passed to a dense output layer.
4. Train it with **backpropagation through time** (BPTT) using mean squared error, Adam (lr = 0.01), batch size 16, 300 epochs and gradient clipping at norm 1.0.
5. **Feed-forward baseline:** 12 inputs, one hidden layer of 44 ReLU units and 1 output, trained with the same settings. Its size (617 parameters) matches the BiRNN (609).
6. Repeat training with 5 random seeds. Report RMSE in thousands of passengers on the training and test data, and compare with a naive baseline that repeats the last month.
7. Plot the test predictions and the training loss of both models.

Fig 1: Structure of the bidirectional RNN

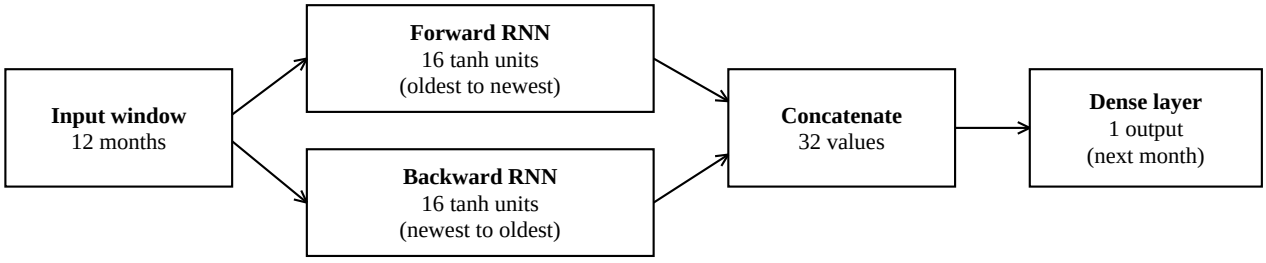
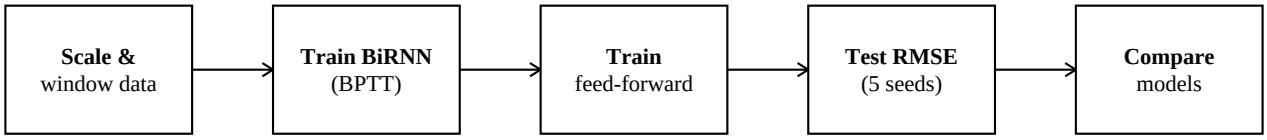


Fig 2: Experiment workflow



Commands / Settings used:

Setting	Bidirectional RNN	Feed-forward (MLP)
Input	12-month window (as a sequence)	12-month window (as 12 inputs)
Hidden layer	16 tanh units per direction	44 ReLU units
Parameters	609	617
Loss / optimizer	MSE / Adam, lr 0.01	MSE / Adam, lr 0.01
Batch / epochs	16 / 300	16 / 300
Train / test split	120 months / 24 months	120 months / 24 months

PROGRAM:

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ---------- Airline Passengers dataset (monthly totals, in thousands, 1949-1960) ----
series = np.array([
    112, 118, 132, 129, 121, 135, 148, 148, 136, 119, 104, 118,
    115, 126, 141, 135, 125, 149, 170, 170, 158, 133, 114, 140,
    145, 150, 178, 163, 172, 178, 199, 199, 184, 162, 146, 166,
    171, 180, 193, 181, 183, 218, 230, 242, 209, 191, 172, 194,
    196, 196, 236, 235, 229, 243, 264, 272, 237, 211, 180, 201,
    204, 188, 235, 227, 234, 264, 302, 293, 259, 229, 203, 229,
    242, 233, 267, 269, 270, 315, 364, 347, 312, 274, 237, 278,
    284, 277, 317, 313, 318, 374, 413, 405, 355, 306, 271, 306,
    315, 301, 356, 348, 355, 422, 465, 467, 404, 347, 305, 336,
    340, 318, 362, 348, 363, 435, 491, 505, 404, 359, 310, 337,
    360, 342, 406, 396, 420, 472, 548, 559, 463, 407, 362, 405,
    417, 391, 419, 461, 472, 535, 622, 606, 508, 461, 390, 432], dtype=float)

LOOK, N_TRAIN = 12, 120                     # window length, months used for training
lo, hi = series[:N_TRAIN].min(), series[:N_TRAIN].max()
scaled = (series - lo) / (hi - lo)          # min-max scaling using training data only


def windows(first, last):
    idx = np.arange(first, last)
    return np.stack([scaled[i - LOOK:i] for i in idx]), scaled[idx][:, None]


Xtr, ytr = windows(LOOK, N_TRAIN)           # 108 training windows
Xte, yte = windows(N_TRAIN, len(series))    # 24 test windows (last 2 years)
print(f"Months: {len(series)}  train windows: {len(Xtr)}  test windows: {len(Xte)}")


# ---------- Model 1: Bidirectional RNN (forward + backward tanh RNN) ----------
class BiRNN:
    def __init__(self, hidden, rng):
        H = self.H = hidden
        s = 1 / np.sqrt(H)
        self.p = {"Wxf": rng.normal(0, s, (1, H)), "Whf": rng.normal(0, s, (H, H)),
                  "bf": np.zeros(H), "Wxb": rng.normal(0, s, (1, H)),
                  "Whb": rng.normal(0, s, (H, H)), "bb": np.zeros(H),
                  "Wd": rng.normal(0, s, (2 * H, 1)), "bd": np.zeros(1)}

    def forward(self, X):
        p, (N, T), H = self.p, X.shape, self.H
        self.X = X
        self.h = [np.zeros((N, H))]                       # h[t+1] after reading x[t]
        for t in range(T):                                # forward in time
            x_t = X[:, t:t + 1]
            self.h.append(np.tanh(x_t @ p["Wxf"] + self.h[-1] @ p["Whf"] + p["bf"]))
```

```
        self.g = [None] * T + [np.zeros((N, H))]          # g[t] after reading x[T-1..t]
        for t in range(T - 1, -1, -1):                    # backward in time
            x_t = X[:, t:t + 1]
            self.g[t] = np.tanh(x_t @ p["Wxb"] + self.g[t + 1] @ p["Whb"] + p["bb"])
        self.z = np.concatenate([self.h[T], self.g[0]], axis=1)
        return self.z @ p["Wd"] + p["bd"]

    def backward(self, dout):                             # backpropagation through time
        p, (N, T), H = self.p, self.X.shape, self.H
        gr = {k: np.zeros_like(v) for k, v in p.items()}
        gr["Wd"], gr["bd"] = self.z.T @ dout, dout.sum(0)
        dz = dout @ p["Wd"].T
        dh = dz[:, :H]
        for t in range(T, 0, -1):                         # forward RNN
            d = dh * (1 - self.h[t] ** 2)
            gr["Wxf"] += self.X[:, t - 1:t].T @ d
            gr["Whf"] += self.h[t - 1].T @ d
            gr["bf"] += d.sum(0)
            dh = d @ p["Whf"].T
        dg = dz[:, H:]
        for t in range(T):                                # backward RNN
            d = dg * (1 - self.g[t] ** 2)
            gr["Wxb"] += self.X[:, t:t + 1].T @ d
            gr["Whb"] += self.g[t + 1].T @ d
            gr["bb"] += d.sum(0)
            dg = d @ p["Whb"].T
        return gr


# ---------- Model 2: traditional feed-forward network (MLP) ----------
class MLP:
    def __init__(self, hidden, rng):
        self.p = {"W1": rng.normal(0, np.sqrt(2 / LOOK), (LOOK, hidden)),
                  "b1": np.zeros(hidden),
                  "W2": rng.normal(0, np.sqrt(2 / hidden), (hidden, 1)),
                  "b2": np.zeros(1)}

    def forward(self, X):
        self.X = X
        self.a = np.maximum(0, X @ self.p["W1"] + self.p["b1"])
        return self.a @ self.p["W2"] + self.p["b2"]

    def backward(self, dout):
        p = self.p
        da = (dout @ p["W2"].T) * (self.a > 0)
        return {"W1": self.X.T @ da, "b1": da.sum(0),
                "W2": self.a.T @ dout, "b2": dout.sum(0)}


def train(model, epochs=300, batch=16, lr=0.01, seed=0):
    rng = np.random.default_rng(seed)
    p = model.p
    m = {k: np.zeros_like(v) for k, v in p.items()}
    v = {k: np.zeros_like(a) for k, a in p.items()}
    t, losses = 0, []
    for ep in range(epochs):
        idx = rng.permutation(len(Xtr))
        for s in range(0, len(idx), batch):
            b = idx[s:s + batch]
            out = model.forward(Xtr[b])
            g = model.backward(2 * (out - ytr[b]) / len(b))     # MSE gradient
            norm = np.sqrt(sum((x ** 2).sum() for x in g.values()))
            scale = min(1.0, 1.0 / (norm + 1e-12))              # gradient clipping
            t += 1
            for k in p:                                         # Adam update
                m[k] = 0.9 * m[k] + 0.1 * g[k] * scale
                v[k] = 0.999 * v[k] + 0.001 * (g[k] * scale) ** 2
                mh = m[k] / (1 - 0.9**t)
                vh = v[k] / (1 - 0.999**t)
                p[k] -= lr * mh / (np.sqrt(vh) + 1e-8)
        losses.append(np.mean((model.forward(Xtr) - ytr) ** 2))
    return losses


def rmse(model, X, y):                      # error in thousands of passengers
```

```
    return np.sqrt(np.mean(((model.forward(X) - y) * (hi - lo)) ** 2))


def count(model):
    return sum(a.size for a in model.p.values())


# ---------- Train both models with 5 different random seeds ----------
res = {"BiRNN": [], "MLP": []}
keep = {}
print(f"BiRNN parameters: {count(BiRNN(16, np.random.default_rng(0)))}, "
      f"MLP parameters: {count(MLP(44, np.random.default_rng(0)))}\n")
print(f"{'Seed':<6}{'BiRNN train':<13}{'BiRNN test':<12}{'MLP train':<11}{'MLP test'}")
print("-" * 50)
for seed in range(5):
    row = {}
    for name, M, hid in (("BiRNN", BiRNN, 16), ("MLP", MLP, 44)):
        model = M(hid, np.random.default_rng(seed))
        loss = train(model, seed=seed)
        row[name] = (rmse(model, Xtr, ytr), rmse(model, Xte, yte))
        res[name].append(row[name])
        if seed == 0:
            keep[name] = (model, loss)
    print(f"{seed:<6}{row['BiRNN'][0]:<13.2f}{row['BiRNN'][1]:<12.2f}"
          f"{row['MLP'][0]:<11.2f}{row['MLP'][1]:.2f}")

print("-" * 50)
for name in res:
    a = np.array(res[name])
    print(f"{name + ' mean':<12}train RMSE = {a[:, 0].mean():.2f} +/- {a[:, 0].std():.2f}   "
          f"test RMSE = {a[:, 1].mean():.2f} +/- {a[:, 1].std():.2f}")
naive = np.sqrt(np.mean(((Xte[:, -1:] - yte) * (hi - lo)) ** 2))
print(f"Naive baseline (repeat last month) test RMSE = {naive:.2f}")
winner = min(res, key=lambda n: np.mean([r[1] for r in res[n]]))
print(f"Lower mean test RMSE: {winner}")

# ---------- Plot: forecasts and training loss (seed 0) ----------
fig, ax = plt.subplots(1, 2, figsize=(9.5, 3.5))
months = np.arange(len(series))
ax[0].plot(months, series, color="gray", label="Actual")
for name, c in (("BiRNN", "tab:blue"), ("MLP", "tab:orange")):
    pred = keep[name][0].forward(Xte) * (hi - lo) + lo
    ax[0].plot(months[N_TRAIN:], pred[:, 0], color=c, label=name + " (test)")
ax[0].axvline(N_TRAIN, ls="--", color="k", lw=.8)
ax[0].set(xlabel="Month index", ylabel="Passengers (thousands)",
          title="Prediction on test years")
for name, c in (("BiRNN", "tab:blue"), ("MLP", "tab:orange")):
    ax[1].plot(keep[name][1], color=c, label=name)
ax[1].set(xlabel="Epoch", ylabel="Training MSE (scaled)", title="Training loss", yscale="log")
for a in ax:
    a.grid(alpha=.3)
    a.legend()
plt.tight_layout()
plt.savefig("img/exp6_forecast.png", dpi=200)
```

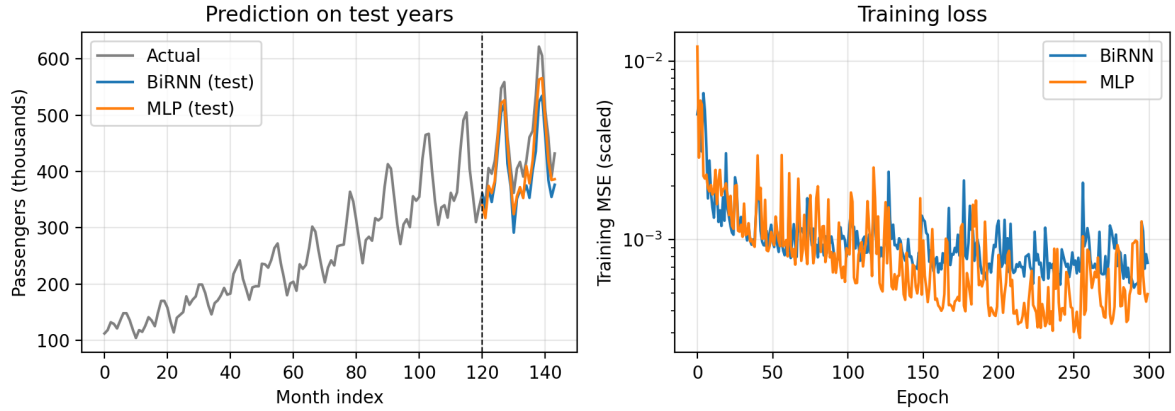
OUTPUT:

Terminal - Bidirectional RNN vs feed-forward network

```
root@ip-10-10-5-10:~# python3 exp6.py
Months: 144  train windows: 108  test windows: 24
BiRNN parameters: 609, MLP parameters: 617

Seed  BiRNN train  BiRNN test  MLP train  MLP test
-------------------------------------------------
0     10.90        57.73       8.91       38.10
1     13.80        57.88       9.39       16.47
2     12.46        59.28       7.55       15.09
3     11.32        36.45       12.43      14.23
4     12.82        29.48       8.43       16.18
-------------------------------------------------
BiRNN mean  train RMSE = 12.26 +/- 1.04   test RMSE = 48.16 +/- 12.62
MLP mean    train RMSE = 9.34 +/- 1.66   test RMSE = 20.01 +/- 9.08
Naive baseline (repeat last month) test RMSE = 51.78
Lower mean test RMSE: MLP
```

Fig 3: Test predictions (seed 0) and training loss of both models



RESULT:

Thus a bidirectional RNN was implemented from scratch with backpropagation through time and compared with a feed-forward network of similar size on the Airline Passengers series. Over 5 seeds the feed-forward network had the lower test error (RMSE 20.01 +/- 9.08 thousand passengers) than the bidirectional RNN (48.16 +/- 12.62), which was close to the naive baseline (51.78). The BiRNN fitted the training years well (RMSE 12.26) but its predictions stayed below the actual values in the last two years, which rise above anything in the training data. So on this task and setup the bidirectional RNN did not outperform the feed-forward network.